Rapport d'equipe
Ajout d'une nouvelle serie STM32Cube dans le pipeline

Projet STM32Cube Preprocessing

20 août 2026

Table des matières

1 Objet du rapport **2**

2 Perimetre technique **2**

3 Fichiers a modifier **2**
- 3.1 Configuration . . . 2
- 3.2 Scripts workflow . . . 2
- 3.3 Workflows CI/CD . . . 2

4 Definition des champs de configuration **2**
- 4.1 Fichier `config_<serie>.json` . . . 2
- 4.2 Fichier `config_all_series.json` . . . 3

5 Source des informations : repos, urls, refs **3**
- 5.1 Commandes de collecte . . . 3
- 5.2 Regles de remplissage . . . 4

6 Procedure d'integration d'une nouvelle serie **4**
- 6.1 Etape 1 : creation du fichier config serie . . . 4
- 6.2 Etape 2 : declaration globale . . . 4
- 6.3 Etape 3 : mapping scripts . . . 4
- 6.4 Etape 4 : mapping batch Python . . . 4
- 6.5 Etape 5 : mapping GitHub Actions . . . 4

7 Validation fonctionnelle **4**
- 7.1 Prepare . . . 4
- 7.2 Split composant . . . 4
- 7.3 Controle couverture et doublons . . . 5
- 7.4 Upload composant . . . 5

8 Test sur une autre machine (self-hosted runner) **5**
- 8.1 Prerequis . . . 5
- 8.2 Installation projet . . . 5
- 8.3 Runner et configuration . . . 5
- 8.4 Test de bout en bout . . . 6

9 Mode migration vers datasources par composant **6**

10 Checklist Go/No-Go **6**

11 Points d'attention **6**
11.1 SHAs incorrects dans `linked_repo_refs` 6
11.2 Serie declaree en config mais absente des `ValidateSet/seriesMap` 7
11.3 Serie absente des workflows 7
11.4 Upload composant sans split prealable 8

1 Objet du rapport

Ce document formalise la procedure standard d'ajout d'une nouvelle serie STM32Cube (exemple : C5) dans le pipeline. Il precise les fichiers a modifier, la signification des champs de configuration, la source des donnees a collecter, l'integration dans l'automatisation, ainsi que les etapes de validation avant mise en production.

Ce rapport est concu pour etre partage avec le document complementaire *Parametres du workflow Manual Auto Update KB et demarrage rapide du runner self-hosted* (`workflow_dispatch_inputs_and_runne`) qui detaille chaque parametre du workflow de declenchement manuel ainsi que l'installation d'un runner self-hosted de test. Les deux documents forment ensemble la documentation de reference remise a l'encadrant.

2 Perimetre technique

Le perimetre couvre :

— la configuration serie,

— l'execution du pipeline (Prepare/Upload),

— le split des issues par composant,

— l'upload vers la KB,

— l'integration dans les workflows GitHub Actions,

— le test sur une autre machine avec self-hosted runner.

3 Fichiers a modifier

3.1 Configuration

— `shared/config/config_c5.json` (nouveau fichier)

— `shared/config/config_all_series.json` (mise a jour)

3.2 Scripts workflow

— `pipeline_Automation/workflow/Run_Single_Series_Full_Pipeline_And_Upload.ps1`

— `pipeline_Automation/workflow/Split_Issues_By_Component.ps1`

— `pipeline_Automation/workflow/Upload_Issues_By_Component.ps1`

— `pipeline_Automation/workflow/run_all_series.py`

3.3 Workflows CI/CD

— `.github/workflows/auto_update_kb.yml`

— `.github/workflows/manual_auto_update_kb.yml`

4 Definition des champs de configuration

4.1 Fichier `config_<serie>.json`

owner

Organisation source des depots (STMicroelectronics).

repos

Liste ordonnee des depots de la serie. Le premier element doit etre le repo parent (exemple : STM32CubeC5), suivi des depots lies (HAL/CMSIS/BSP/Middleware).

data_dir
Dossier des artefacts data intermediaires (valeur standard : `data`).
repos_dir
Dossier local des clones (valeur standard : `GitHub_repos`).
auto_clone_missing_repos
Activation du clonage automatique des depots manquants.
repo_clone_urls
Dictionnaire `repo -> URL git` pour les depots relies.
linked_repo_refs
Dictionnaire `repo -> SHA` correspondant aux refs des sous-modules pour la release cible. Ce champ est essentiel pour la reproductibilite.
commit_pr_scope
Strategie de liaison des commits/PR (valeur recommandee : `linked_or_merged`).
github_token_env
Nom de variable d'environnement contenant le token GitHub.
rate_limits
Temporisations API (issues/comments) pour stabiliser les appels.
ingest_version
Version logique de la configuration d'ingestion.

4.2 Fichier `config_all_series.json`

all_series
Liste des series actives.
series_config_files
Mapping `Serie -> fichier config`.
kb_datasource_ids
IDs des datasources par serie (issues, files, diagnostic, resolver).
kb_id
Identifiant de la Knowledge Base cible.

5 Source des informations : repos, urls, refs

Les informations sont extraites du repo parent de la serie, positionne sur la release cible.

5.1 Commandes de collecte

```
git clone --recursive https://github.com/STMicroelectronics/STM32CubeC5.git
cd STM32CubeC5
git checkout <tag_cible>
git submodule update --init --recursive

# URLs des sous-modules
git config -f .gitmodules --get-regexp "submodule\..*\.url"

# SHAs de references
git submodule status --recursive

# Paths des sous-modules
git config -f .gitmodules --get-regexp "submodule\..*\.path"
```

5.2 Regles de remplissage

— `repos[0]` = repo parent de la serie.
— `repo_clone_urls` renseigne les depots a cloner.
— `linked_repo_refs` renseigne les SHAs de la release cible.
— Exclure les depots non relies au perimetre de la serie.

6 Procedure d'integration d'une nouvelle serie

6.1 Etape 1 : creation du fichier config serie

Creer `shared/config/config_c5.json` a partir d'un modele proche (`config_c0.json` ou `config_g4.json`) et renseigner les champs `repos`, `repo_clone_urls`, `linked_repo_refs`.

6.2 Etape 2 : declaration globale

Mettre a jour `shared/config/config_all_series.json` :
— ajout dans `all_series`,
— mapping dans `series_config_files`,
— entree dans `kb_datasource_ids`.

6.3 Etape 3 : mapping scripts

Ajouter la serie dans tous les `ValidateSet` et `seriesMap` :
— code serie : `C5`
— slug serie : `stm32cubec5`
— repo parent : `STM32CubeC5`
— config : `shared/config/config_c5.json`

6.4 Etape 4 : mapping batch Python

Dans `run_all_series.py`, ajouter `c5` dans `SERIES_CONFIGS`.

6.5 Etape 5 : mapping GitHub Actions

Ajouter la serie C5 dans :
— les listes de series autorisees,
— les listes par defaut (si desired),
— les normalisations d'inputs.

7 Validation fonctionnelle

7.1 Prepare

```
powershell -ExecutionPolicy Bypass -File .\pipeline_Automation\workflow\
    Run_Single_Series_Full_Pipeline_And_Upload.ps1 -Series C5 -Mode Prepare
```

7.2 Split composant

```
powershell -ExecutionPolicy Bypass -File .\pipeline_Automation\workflow\
    Split_Issues_By_Component.ps1 -Series C5
```

7.3 Controle couverture et doublons

```
python -m pipeline.delivery.validate_issues_by_component_split --summary "datasets/07
    _delivery/st_ready/by_series/stm32cubec5/issues_json/by_component/
    summary_by_component_c5.json"
```

7.4 Upload composant

```
powershell -ExecutionPolicy Bypass -File .\pipeline_Automation\workflow\
    Upload_Issues_By_Component.ps1 -Series C5 -DryRun -RemoteUser "<email_st>"
powershell -ExecutionPolicy Bypass -File .\pipeline_Automation\workflow\
    Upload_Issues_By_Component.ps1 -Series C5 -RemoteUser "<email_st>"
```

8 Test sur une autre machine (self-hosted runner)

8.1 Prerequis

— Windows 10/11

— Git

— Python 3.12+

— acces reseau GitHub/ST

8.2 Installation projet

```
git clone https://github.com/<org-ou-user>/PFE_Chatbot_STM32Cube.git
cd PFE_Chatbot_STM32Cube
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

8.3 Runner et configuration

— Declarer un self-hosted runner depuis les settings du repo, en suivant la documentation officielle GitHub : https://docs.github.com/en/actions/how-tos/manage-runners/self-hosted-runners/add-runners.

— Lancer le runner en service.

— Verifier le label `self-hosted`.

— Activer `git core.longpaths=true`.

— Configurer secrets : `ST_GITHUB_ANALYZER_API_KEY`, `ST_CHATGPT_API_KEY`, `ST_REMOTE_USER`.

Important – clone du depot vs reconfiguration du runner. Lorsqu'une nouvelle machine clone ce depot, certaines etapes ont deja ete realisees (scripts, configuration pipeline, dossier `actions-runner` present dans l'arborescence clonee). En revanche, le runner self-hosted lui-meme n'est **pas** reutilisable tel quel : il doit etre reconfigure sur la nouvelle machine. Pour cela :

1. Supprimer le contenu du dossier `actions-runner` (le dossier provient de la machine d'origine et contient une configuration/registration qui lui est propre).
2. Reprendre les instructions a partir de l'**etape 6** du document GitHub officiel cite ci-dessus (telechargement et configuration du runner).

3. Executer ces commandes depuis une console **PowerShell** ou **CMD** Windows, et non depuis Git Bash (le script de configuration du runner n'est pas garanti de fonctionner correctement sous Git Bash).
4. Lors de la (re)configuration (`config.cmd`), il est recommande de conserver les options par defaut proposees (nom de runner, groupe, labels, dossier de travail), sauf besoin explicite de les modifier.

8.4 Test de bout en bout

```
powershell -ExecutionPolicy Bypass -File .\pipeline_Automation\workflow\
    Run_Single_Series_Full_Pipeline_And_Upload.ps1 -Series C5 -Mode Full -
    EnableIssuesByComponentFlow -ComponentIssuesOnly -RemoteUser "<email_st>"
```

9 Mode migration vers datasources par composant

Pour remplacer le datasource global issues par des datasources `issues_<SERIE>_<COMPONENT>` :

— activer `EnableIssuesByComponentFlow`,

— activer `ComponentIssuesOnly`.

10 Checklist Go/No-Go

— config serie creee et controlee,

— config globale mise a jour,

— mappings scripts a jour,

— mappings workflows a jour,

— phase Prepare validee,

— split composant valide,

— validation anti-doublons OK,

— upload dry-run OK,

— upload reel OK,

— datasource IDs traces dans `config_all_series.json`.

11 Points d'attention

Cette section detaille chacun des quatre risques operationnels identifies pour l'ajout d'une nouvelle serie. Pour chaque point : la notion concernee, pourquoi c'est un risque, un exemple concret, et comment le detecter/corriger.

11.1 SHAs incorrects dans `linked_repo_refs`

Notion. Le champ `linked_repo_refs` du fichier `config_<serie>.json` associe chaque sous-depot (HAL, CMSIS, BSP, Middleware) au SHA de commit exact correspondant a la release ciblee du depot parent. C'est ce SHA, et non une branche mobile, qui garantit que le contenu ingere correspond a une version figee et tracable.

Pourquoi c'est un risque. Si un SHA est incorrect (copie depuis la mauvaise branche, depot desynchronise, ou oubli de mise a jour apres un changement de release), le pipeline ingere un etat du sous-depot qui ne correspond pas a la release officielle. Les documents generes referencent alors un contenu different de celui reellement publie, ce qui casse la *reproductibilite* : deux executions du pipeline a des dates differentes, ou par deux personnes differentes, ne produisent plus le meme resultat pour la meme release nominale.

Exemple concret. Un SHA laisse sur la branche `main` au lieu du tag `v1.2.0` de la release : si le sous-depot recoit de nouveaux commits apres l'integration, la prochaine execution du pipeline changera silencieusement les donnees ingerees, sans que la version de la serie declaree ait change.

Detection et correction. Toujours extraire les SHAs avec `git submodule status -recursive` apres avoir positionne le depot parent exactement sur le tag/commit de la release cible (voir section 5.1 sur les commandes de collecte), jamais a la main ni depuis une branche en mouvement.

11.2 Serie declaree en config mais absente des `ValidateSet`/`seriesMap`

Notion. `ValidateSet` est une contrainte PowerShell qui restreint les valeurs acceptees pour un parametre de script (ex : liste figee de codes serie). `seriesMap` est la table interne (PowerShell ou Python) qui relie un code serie (ex : `C5`) a son fichier de configuration, son slug et son depot parent.

Pourquoi c'est un risque. Declarer une serie uniquement dans `config_all_series.json` ne suffit pas si un script utilise encore une liste `ValidateSet` ou une table `seriesMap` codee en dur : le script refuse la serie ou echoue en lui cherchant un fichier de configuration inexistant, meme si la configuration JSON globale est correcte.

Etat verifie pour la serie C5 (mise a jour de ce rapport). Une verification du code a ete effectuee. Les scripts principaux `Run_Single_Series_Full_Pipeline_And_Upload.ps1`, `Split_Issues_By_Compo` `Upload_Issues_By_Component.ps1` et `Upload_Series_All_Categories.ps1` construisent leur `seriesMap` *dynamiquement* a partir de `shared/config/config_all_series.json` : comme `STM32CubeC5` et `config_c5.json` y sont bien enregistres, **ce risque est corrige pour ces quatre scripts et la serie C5 est utilisable directement**.

Toutefois, deux scripts auxiliaires plus anciens utilisent encore des tables *statiques* qui n'incluent pas C5 et devront etre mis a jour manuellement avant utilisation avec cette serie :

— `pipeline_Automation/workflow/run_all_series.py` (dictionnaire `SERIES_CONFIGS` code en dur, C5 absent),

— `pipeline_Automation/workflow/reexport_delivery.py` (meme dictionnaire `SERIES_CONFIGS` statique, C5 absent),

— `pipeline_Automation/workflow/Upload_Multi_Series_All_Categories.ps1` (liste `ValidateSet` figee, C5 absent).

Ces trois scripts restent **a risque** tant qu'ils ne sont pas mis a jour : leur usage avec `-Series C5` (ou `-series c5`) echouera avec une erreur de type « config introuvable » ou « valeur non autorisee ».

Detection et correction. Avant d'utiliser un script pour une serie nouvellement ajoutee, verifier s'il lit `config_all_series.json` dynamiquement (pas de risque) ou s'il possede une liste/dictionnaire de series code en dur (risque residuel, necessite un ajout manuel de la serie dans ce fichier).

11.3 Serie absente des workflows

Notion. Les workflows GitHub Actions (`.github/workflows/auto_update_kb.yml` et `.github/workflows/m` exposent les series disponibles via leurs parametres `workflow_dispatch` (voir le document complementaire sur les parametres du workflow). Meme si une serie est correctement configuree cote pipeline, elle doit aussi etre acceptee par la logique de normalisation des inputs du workflow.

Pourquoi c'est un risque. Si la serie n'est pas prise en compte dans cette normalisation (ou dans une liste par defaut du workflow), il devient impossible de la declencher depuis l'interface *Actions* de GitHub, meme en executant le script localement avec succes : le workflow rejettera l'entree ou ne l'inclura pas dans la matrice de jobs.

Etat verifie pour C5. Les workflows lisent egalement `config_all_series.json` pour deriver la liste des series autorisees (meme mecanisme que `seriesMap`), donc C5 y est normalement deja acceptee. Il reste toutefois recommande de faire un essai de declenchement manuel avec `series=C5` et `mode=Prepare` pour confirmer avant un usage en production.

Detection et correction. Lancer le workflow *Manual Auto Update KB* avec la nouvelle serie en mode `Prepare` et verifier que le job correspondant apparait dans la matrice, avant de tenter un mode `Full`.

11.4 Upload composant sans split prealable

Notion. Le « split par composant » (script `Split_Issues_By_Component.ps1`) decoupe le datasource « issues » monolithique d'une serie en plusieurs datasources plus petits, un par composant logiciel (HAL, BSP, Middleware, etc.), et genere en meme temps un fichier `summary_by_component_<serie>.json` qui recapitule la repartition et sert de reference pour le controle anti-doublons.

Pourquoi c'est un risque. Le script `Upload_Issues_By_Component.ps1` attend en entree ce fichier `summary_by_component` pour savoir quels composants uploader et avec quels identifiants de datasource. Si l'upload par composant est lance sans avoir prealablement execute le split, ce fichier de synthese est absent et l'upload ne peut pas determiner la liste des composants a traiter : il echoue immediatement ou traite une liste vide.

Exemple concret. Executer directement `Upload_Issues_By_Component.ps1 -Series C5` sans avoir lance au prealable `Split_Issues_By_Component.ps1 -Series C5` produit une erreur de fichier manquant plutot qu'un upload partiel silencieux.

Detection et correction. Toujours respecter l'ordre : (1) export delivery standard, (2) `Split_Issues_By_Co` (3) controle du resume via `validate_issues_by_component_split`, (4) uniquement ensuite `Upload_Issues_By_` Cet ordre est deja formalise dans la section « Validation fonctionnelle » de ce document.